

Absolutely. Below is a **complete answer for Parts 1–6**, with comments explaining what each line is doing. I'll keep the code close to the level of your reading guides rather than making it unnecessarily advanced.

The problem combines the Week 2 material on classes, custom exceptions, and specific exception handling with the Week 3 material on lambda, NumPy, Pandas, cosine similarity, and train/test splitting.

Part 1 — Ticket Class and Validation

```
# We create our own exception type.
# It inherits from Python's built-in Exception class.
class InvalidTicketError(Exception):
    pass

# This class represents one support ticket.
class SupportTicket:

    # __init__ runs automatically when we create a SupportTicket object.
    def __init__(self, ticket_id, message, category, priority):

        # Check if ticket_id is empty or only contains spaces.
        if not ticket_id.strip():
            raise InvalidTicketError("Ticket ID is empty")

        # Check if message is empty or only contains spaces.
        if not message.strip():
            raise InvalidTicketError("Message is empty")

        # Priority must be from 1 to 5.
        if priority < 1 or priority > 5:
            raise InvalidTicketError("Priority must be between 1 and 5")

        # Store the data inside the object.
        self.ticket_id = ticket_id
        self.message = message
        self.category = category
        self.priority = priority

    # This method returns a readable summary of the ticket.
    def summary(self):
```

```

        return f"{self.ticket_id}: [{self.category}] {self.message} | priority={self.priority}"

# -----
# TEST 1: Valid ticket
# -----

ticket1 = SupportTicket(
    "T100",
    "I love Python",
    "billing",
    1
)

# Call the summary() method.
print(ticket1.summary())

# -----
# TEST 2: Invalid ticket
# -----

try:
    # The message contains only spaces,
    # so this should raise InvalidTicketError.
    ticket2 = SupportTicket(
        "T101",
        " ",
        "billing",
        1
    )

except InvalidTicketError as e:
    # Instead of getting a traceback,
    # print a friendly error message.
    print(f"Invalid ticket: {e}")

```

What you should understand

The important pattern is:

```

class InvalidTicketError(Exception):
    pass

```

then:

```
raise InvalidTicketError("Message is empty")
```

```
then:
```

```
try:
```

```
...
```

```
except InvalidTicketError as e:
```

```
...
```

That is the custom-exception pattern covered in your Week 2 reading.

Part 2 — Cleaning the Data with Pandas

This is the part you were working on.

The key thing to remember is:

```
df = pd.DataFrame(raw_tickets)
```

automatically turns the list of dictionaries into columns.

```
import pandas as pd
```

```
# -----
```

```
# Raw data
```

```
# -----
```

```
raw_tickets = [
```

```
    {"id": "T101", "message": "Payment failed", "category": "Billing", "priority": 3},
```

```
    {"id": "T102", "message": "Cannot login", "category": "technical", "priority": 4},
```

```
    {"id": "T103", "message": "Refund not received", "category": "BILLING", "priority": None},
```

```
    {"id": "T104", "message": "Password reset please", "category": "Technical", "priority": 2},
```

```
    {"id": "T104", "message": "Password reset please", "category": "Technical", "priority": 2},
```

```
    {"id": "T105", "message": "app keeps crashing", "category": "TECHNICAL", "priority": 5},
```

```
]
```

```
# Convert the list of dictionaries into a DataFrame.
```

```
# Pandas automatically creates columns from the dictionary keys.
```

```
df = pd.DataFrame(raw_tickets)
```

```
print("BEFORE CLEANING:")
print(df)
```

```
# -----
```

```
# 1. Remove extra whitespace
```

```
# -----
```

```
# .str.strip() removes spaces from the beginning and end.
```

```
df["id"] = df["id"].str.strip()
```

```
df["message"] = df["message"].str.strip()
```

```
df["category"] = df["category"].str.strip()
```

```
# -----
```

```
# 2. Normalize category
```

```
# -----
```

```
# Convert all category values to lowercase.
```

```
# Example:
```

```
# "Billing" -> "billing"
```

```
# "BILLING" -> "billing"
```

```
# "Technical" -> "technical"
```

```
df["category"] = df["category"].str.lower()
```

```
# -----
```

```
# 3. Fill missing priority
```

```
# -----
```

```
# Find the average priority.
```

```
# The missing value is ignored when calculating the mean.
```

```
mean_priority = df["priority"].mean()
```

```
# Replace the missing priority with the mean.
```

```
df["priority"] = df["priority"].fillna(mean_priority)
```

```
# -----
```

```
# 4. Remove duplicate rows
```

```
# -----
```

```
# The T104 row appears twice.
```

```
# drop_duplicates() keeps the first copy and removes the duplicate.
```

```

df = df.drop_duplicates()

# -----
# 5. Add message_length
# -----

# .str.len() counts the number of characters in each message.
df["message_length"] = df["message"].str.len()

# -----
# Show cleaned data
# -----

print("\nAFTER CLEANING:")
print(df)

# describe() gives statistical information
# about the numeric columns.
print("\nDESCRIBE:")
print(df.describe())

```

The important Pandas commands to memorize

```

df = pd.DataFrame(raw_tickets)

Create DataFrame.

df["name"] = df["name"].str.strip()

Remove whitespace.

df["name"] = df["name"].str.lower()

Normalize text.

df["score"] = df["score"].fillna(df["score"].mean())

Fill missing values using the mean.

df = df.drop_duplicates()

Remove duplicate rows.

df["length"] = df["text"].str.len()

```

Create a column based on string length.

```
df.describe()
```

Get a statistical summary.

These operations are directly covered in the Week 3 Pandas cleaning material.

Part 3 — Lambda + Priority Ranking

Here we're going to sort the cleaned tickets.

The requirement is:

1. Highest priority first.
2. If priority is equal, longest message first.

```
# Convert the DataFrame into a list of dictionaries.
```

```
# Each dictionary represents one ticket.
```

```
records = df.to_dict("records")
```

```
# Sort the records.
```

```
#
```

```
# lambda x: creates a small temporary function.
```

```
#
```

```
# x["priority"] is the first sorting value.
```

```
# x["message_length"] is the second sorting value.
```

```
#
```

```
# reverse=True means highest values come first.
```

```
ranked = sorted(  
    records,  
    key=lambda x: (x["priority"], x["message_length"]),  
    reverse=True  
)
```

```
# Print the ranked ticket IDs.
```

```
print("Ranked ticket IDs:")
```

```
for ticket in ranked:
```

```
    print(ticket["id"])
```

Why this works

Suppose we have:

```
priority = 5  
message_length = 20
```

and:

```
priority = 5  
message_length = 30
```

Because priority is tied, the second ticket comes first because its message is longer.

The lambda:

```
lambda x: (x["priority"], x["message_length"])
```

creates the sorting key.

This follows the Week 3 material on using lambda with `sorted()`.

Part 4 — NumPy Vectorized Priority Adjustment

Now we use NumPy.

The rule is:

Add 1 to every priority, but never allow a priority above 5.

```
import numpy as np
```

```
# Get the priority column from the DataFrame.  
# Convert it into a NumPy array.  
priorities = np.array(df["priority"])
```

```
# Add 1 to every value and cap everything at 5.  
#
```

```
# np.clip(array, minimum, maximum)
# means:
# values below minimum -> minimum
# values above maximum -> maximum
#
# Example:
# 2 -> 3
# 3 -> 4
# 4 -> 5
# 5 -> 5
adjusted_priorities = np.clip(priorities + 1, 1, 5)
```

```
# Print the original priorities.
print("Original priorities:")
print(priorities)
```

```
# Print the adjusted priorities.
print("Adjusted priorities:")
print(adjusted_priorities)
```

```
# Calculate the average adjusted priority.
average_priority = adjusted_priorities.mean()

print("Average adjusted priority:", average_priority)
```

```
# Vectorized NumPy operations work on the whole array at once,
# avoiding a Python loop for every individual element.
```

The important part is:

```
adjusted_priorities = np.clip(priorities + 1, 1, 5)
```

You could also use `np.where()`, but `np.clip()` is simpler for this particular problem.

Your guide specifically discusses vectorization, broadcasting, `np.where()`, and NumPy operations over entire arrays.

Part 5 — Find the Most Similar Ticket

Now we're doing vector similarity.

The reading guide explains cosine similarity as comparing the direction of vectors, and demonstrates `sklearn.metrics.pairwise.cosine_similarity`.

```
import numpy as np

from sklearn.metrics.pairwise import cosine_similarity

# Each row represents the vector for one ticket.
ticket_vectors = np.array([
    [0.10, 0.20, 0.90],
    [0.80, 0.10, 0.20],
    [0.15, 0.25, 0.85],
    [0.70, 0.20, 0.30],
    [0.20, 0.30, 0.80]
])

# This represents the user's new query.
query_vector = np.array([
    0.12,
    0.22,
    0.88
])

# Calculate cosine similarity between every ticket
# and the query vector.
#
# reshape(1, -1) changes the query from:
# [0.12, 0.22, 0.88]
#
# into:
# [[0.12, 0.22, 0.88]]
#
# which makes it a 2D row vector.
similarities = cosine_similarity(
    ticket_vectors,
    query_vector.reshape(1, -1))
```

```
).flatten()
```

```
# Print every ticket's similarity.  
for i, score in enumerate(similarities):  
    print(f"Ticket {i + 1}: {score:.4f}")
```

```
# np.argmax() gives the position of the largest similarity score.  
best_index = np.argmax(similarities)
```

```
# Convert the index into a human-friendly ticket number.  
best_ticket = best_index + 1
```

```
print("\nMost similar ticket:")  
print(f"Ticket {best_ticket}")
```

```
# Print the actual similarity score.  
print(f"Similarity: {similarities[best_index]:.4f}")
```

Why `argmax()`?

Suppose:

```
similarities = [0.91, 0.43, 0.97, 0.51, 0.88]
```

Then:

```
np.argmax(similarities)
```

returns:

```
2
```

because index `2` contains `0.97`, the largest number.

Remember that Python uses zero-based indexing, so:

```
index 0 → first  
index 1 → second  
index 2 → third
```

This is very similar to the RAG example in your reading guide, where cosine similarities are calculated and `np.argmax()` is used to retrieve the best matching document.

Part 6 — Train/Test Split

This one is important because it's part of the Week 3 material on evaluating models fairly.

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
# Create a small dataset.
```

```
data = pd.DataFrame({
    "message": [
        "payment failed",
        "cannot login",
        "refund missing",
        "password reset",
        "card charged twice",
        "app keeps crashing",
        "wrong invoice",
        "server is down",
        "need a refund",
        "cannot access account"
    ],
    "category": [
        "billing",
        "technical",
        "billing",
        "technical",
        "billing",
        "technical",
        "billing",
        "technical",
        "billing",
        "technical"
    ]
})
```

```

# Split the dataset.
#
# test_size=0.2 means 20% goes to the test set.
#
# random_state=42 makes the split reproducible.
#
# stratify=data["category"] makes sure the categories
# remain proportionally represented in train and test.
train_set, test_set = train_test_split(
    data,
    test_size=0.2,
    random_state=42,
    stratify=data["category"]
)

# Print the sizes.
print("Training set size:", len(train_set))
print("Test set size:", len(test_set))

# Show the two sets.
print("\nTRAINING SET:")
print(train_set)

print("\nTEST SET:")
print(test_set)

```

Why split the data?

The basic idea is that the model should be trained on one set of examples and evaluated on examples it did not train on. Otherwise, you can get an overly optimistic evaluation because the model may have effectively memorized the training data.

Your Week 3 guide specifically covers `train_test_split`, `test_size`, `random_state`, and `stratify`, as well as overfitting and data leakage.

Full Practice Answer

For studying, here is what the overall structure looks like when you put the concepts together:

```
# =====  
# PART 1 — OOP + EXCEPTION  
# =====
```

```
class InvalidTicketError(Exception):  
    pass
```

```
class SupportTicket:  
    def __init__(self, ticket_id, message, category, priority):  
  
        if not ticket_id.strip():  
            raise InvalidTicketError("Ticket ID is empty")  
  
        if not message.strip():  
            raise InvalidTicketError("Message is empty")  
  
        if priority < 1 or priority > 5:  
            raise InvalidTicketError("Priority must be between 1 and 5")  
  
        self.ticket_id = ticket_id  
        self.message = message  
        self.category = category  
        self.priority = priority  
  
    def summary(self):  
        return f"{self.ticket_id}: [{self.category}] {self.message} | priority={self.priority}"
```

```
ticket1 = SupportTicket("T100", "I love Python", "billing", 1)  
print(ticket1.summary())
```

```
try:  
    ticket2 = SupportTicket("T101", " ", "billing", 1)  
except InvalidTicketError as e:  
    print(f"Invalid ticket: {e}")
```

```
# =====  
# PART 2 — PANDAS  
# =====
```

```
import pandas as pd
```

```

raw_tickets = [
    {"id": " T101 ", "message": " Payment failed ", "category": "Billing", "priority": 3},
    {"id": "T102", "message": "Cannot login", "category": " technical ", "priority": 4},
    {"id": "T103", "message": "Refund not received", "category": "BILLING", "priority": None},
    {"id": "T104", "message": "Password reset please", "category": "Technical", "priority": 2},
    {"id": "T104", "message": "Password reset please", "category": "Technical", "priority": 2},
    {"id": "T105", "message": " app keeps crashing ", "category": "TECHNICAL", "priority": 5},
]

```

```

# Automatically creates columns from dictionary keys.
df = pd.DataFrame(raw_tickets)

```

```

# Clean whitespace.
df["id"] = df["id"].str.strip()
df["message"] = df["message"].str.strip()
df["category"] = df["category"].str.strip()

```

```

# Normalize categories.
df["category"] = df["category"].str.lower()

```

```

# Fill missing priority with mean.
df["priority"] = df["priority"].fillna(df["priority"].mean())

```

```

# Remove duplicate rows.
df = df.drop_duplicates()

```

```

# Create message length column.
df["message_length"] = df["message"].str.len()

```

```

print(df)
print(df.describe())

```

```

# =====
# PART 3 — LAMBDA
# =====

```

```

records = df.to_dict("records")

```

```

ranked = sorted(
    records,
    key=lambda x: (x["priority"], x["message_length"]),
    reverse=True
)

```

```
print("\nRanked tickets:")
```

```
for ticket in ranked:  
    print(ticket["id"])
```

```
# =====  
# PART 4 — NUMPY  
# =====
```

```
import numpy as np
```

```
priorities = np.array(df["priority"])
```

```
# Add 1 to every value, but never go above 5.  
adjusted_priorities = np.clip(priorities + 1, 1, 5)
```

```
print("\nAdjusted priorities:")  
print(adjusted_priorities)
```

```
print("Average:", adjusted_priorities.mean())
```

```
# =====  
# PART 5 — COSINE SIMILARITY  
# =====
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
ticket_vectors = np.array([  
    [0.10, 0.20, 0.90],  
    [0.80, 0.10, 0.20],  
    [0.15, 0.25, 0.85],  
    [0.70, 0.20, 0.30],  
    [0.20, 0.30, 0.80]  
])
```

```
query_vector = np.array([0.12, 0.22, 0.88])
```

```
similarities = cosine_similarity(  
    ticket_vectors,  
    query_vector.reshape(1, -1)  
).flatten()
```

```

print("\nSimilarity scores:")

for i, score in enumerate(similarities):
    print(f'Ticket {i + 1}: {score:.4f}')

best_index = np.argmax(similarities)

print(
    f'Best match: Ticket {best_index + 1} '
    f'({similarities[best_index]:.4f})'
)

# =====
# PART 6 — TRAIN/TEST SPLIT
# =====

from sklearn.model_selection import train_test_split

data = pd.DataFrame({
    "message": [
        "payment failed",
        "cannot login",
        "refund missing",
        "password reset",
        "card charged twice",
        "app keeps crashing",
        "wrong invoice",
        "server is down",
        "need a refund",
        "cannot access account"
    ],
    "category": [
        "billing",
        "technical",
        "billing",
        "technical",
        "billing",
        "technical",
        "billing",
        "technical",
        "billing",
        "technical"
    ]
})

```

```

        "technical"
    ]
})

train_set, test_set = train_test_split(
    data,
    test_size=0.2,
    random_state=42,
    stratify=data["category"]
)

print("\nTraining size:", len(train_set))
print("Testing size:", len(test_set))

```

What I would memorize for the exam

Don't try to memorize the entire solution. Memorize these **patterns**:

```

# Custom exception
class MyError(Exception):
    pass
# Raise + catch
raise MyError("something went wrong")

try:
    ...
except MyError as e:
    print(e)
# List of dictionaries → DataFrame
df = pd.DataFrame(data)
# Pandas cleaning
df["column"] = df["column"].str.strip()
df["column"] = df["column"].str.lower()
df["column"] = df["column"].fillna(df["column"].mean())
df = df.drop_duplicates()
# Lambda + sorted
sorted(data, key=lambda x: x["score"], reverse=True)
# NumPy vectorization
result = np.clip(array + 1, 1, 5)
# Cosine similarity
similarities = cosine_similarity(
    vectors,
    query.reshape(1, -1)
).flatten()

```

```
best = np.argmax(similarities)
# Train/test split
train, test = train_test_split(
    df,
    test_size=0.2,
    random_state=42,
    stratify=df["category"]
)
```

Those patterns cover the major skills your reading guides emphasize: Python foundations, OOP and exceptions, lambda/higher-order functions, vectorized NumPy, Pandas cleaning, cosine similarity, and train/test splitting.